2024 33rd International Conference on Computer Communications and Networks (ICCCN) | 979-8-3503-8461-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICCCN61486.2024.10637599

QATFP-YOLO: Optimizing Object Detection on Non-GPU Devices with YOLO Using Quantization-Aware Training and Filter Pruning

Gift Idama*, Yifan Guo*, and Wei Yu*
*Department of Computer and Information Sciences,
Towson University, Towson, MD, USA 21252
Emails: gift.idama95@gmail.com, {yguo, wyu}@towson.edu

***Abstract*—Object detection is significant in real-world applications, including self-driving cars, surveillance systems, and vision-enabled robotic systems, among others. Despite the success of benchmark deep learning-based approaches, like YOLO, achieving high detection accuracy, they are typically computationally intensive and require GPUs to achieve optimal performance, preventing them from being widely deployed on low-power end-user devices. Particularly, when deploying these models on non-GPU devices, their inference speed is significantly degraded due to the lack of GPU support. To this end, in this paper, we propose an optimized object detection model called QATFP-YOLO (Quantization-Aware Training and Filter Pruning on YOLO), aiming to enhance inference speed on non-GPU devices, which could be trained and inferred on local end-user devices without GPU support. To reduce the computation complexity, we propose two optimized training strategies based on our QATFP-YOLO model by considering: (i) model quantization technique that reduces the model size and memory usage without sacrificing accuracy and (ii) filter pruning technique that removes redundant parameters from the model, further reducing memory usage and inference time. By evaluating the performance on a real smartphone, we find our QATFP-YOLO model achieves exceptional inference speeds, reaching approximately 88 frames per second, notably surpassing traditional YOLO-Lite models by over fourfold.**

***Index Terms*—Quantization Aware Training, Filter Pruning, Object Detection, YOLO, Edge Intelligence, Internet of Things**

I. INTRODUCTION

Object detection is paramount in real-world applications, involving identifying both the presence (localization) and type (classification) of objects within images or video frames [1], [2]. It plays an essential role in various smart-world systems empowered by Internet of Things (IoT) technology today, including autonomous vehicles, as implemented in self-driving cars (Tesla, Waymo, etc.), video or UAV surveillance systems [3] in places (e.g., airports, traffic cams, homes), health care systems [4], as well as robotics that enables robots to perceive and interact with objects in its environment [1], [5]–[7].

Various approaches to object detection include traditional techniques like Haar cascades [8], and deep learning-based (particularly Convolutional Neural Networks (CNNs)-based) approaches, including Faster R-CNN [9], YOLO [10], SSD [11], etc., which have proven highly effective for object detection. These deep learning models can handle object detection tasks effectively by employing neural networks to learn from the input and extract features with rich data inputs and high computing resources.

However, these models, which are typically computationally intensive, require GPUs to achieve optimal performance, preventing them from being widely deployed on low-power end-user devices that are widely used in typical IoT systems. To deal with this issue, Huang *et al.* [12] proposed YOLO-Lite, an optimized version with lightweight architectures tailored for non-GPU devices. YOLO-Lite attained an inference speed of 21 frames per second (FPS) on non-GPU devices but with a negligible drop in inference accuracy. Nonetheless, there is still room for further enhancements.

Also, other techniques, such as model quantization and Huffman coding, can be used to reduce the nueral network size and increase its speed on non-GPU devices or embedded systems. For example, Han *et al.* [13] proposed a method called "deep compression" that addresses the computational and memory limitations of neural networks on embedded systems. This method consists of a 3-stage pipeline: pruning, dynamic quantization, and Huffman coding, which collectively reduce the storage size of neural networks without compromising accuracy. Their approach involves selectively removing connections, reducing bit size used to represent weights, and applying efficient coding techniques, resulting in significant storage reduction, making them suitable for deployment on resource-constrained systems.

Inspired by this, in this paper, we aim to design an optimized object detection model to achieve a higher inference speed on non-GPU devices based on a benchmark YOLO model called QATFP-YOLO. Particularly, we develop two optimization strategies applying our proposed model: (i) a 2-stage approach featuring Quantization-Aware Training (QAT) and (ii) a 3-stage approach featuring QAT and Filter Pruning (FT). In our 2-stage approach, our model utilizes smaller feature maps and fewer layers than the original TinyYOLO architecture, aiming to reduce computational complexity without sacrificing accuracy. The 2-stage pipeline involves model compression and quantization to reduce model size and memory usage. In our 3-stage approach, we further apply filter pruning techniques to reduce computational complexity based on our 2-stage approach. To assess the performance, we train the

979-8-3503-8461-1/24/$31.00 ©2024 IEEE
Authorized licensed use limited to: Universidad de Sevilla. Downloaded on April 18,2026 at 11:40:52 UTC from IEEE Xplore. Restrictions apply.

model on the PascalVOC dataset and deploy our model on a real model device with an AI Benchmark app to measure the inference speed. Remarkably, our QATFP-YOLO model achieves exceptional inference speeds, reaching approximately 88 FPS, surpassing traditional YOLO-Lite models by over fourfold.

The main contributions of this paper are listed below.

- We propose an optimized YOLO model, called QATFP-YOLO, which uses smaller feature maps and fewer layers to reduce computational complexity, bringing high inference speed while maintaining high accuracy.
- We design two optimization strategies by considering both the model quantization technique that reduces the model size and memory usage without sacrificing accuracy and the filter pruning technique that removes redundant parameters from the model, further reducing memory usage and inference time.
- We validate the inference performance on a real model device with benchmark datasets and found exceptional inference speeds of our proposed model.

The remainder of this paper is organized as follows. In Section II, we review and discuss different deep learning-based approaches for object detection. In Section III, we introduce our approach in detail. In Section IV, we demonstrate and discuss evaluation results. Section V concludes the paper.

II. Background and Related Work

Faster CNN: Faster R-CNN [9] is a region-based CNN that improves the speed and accuracy of object detection. The model includes a region proposal network (RPN) that engages candidate object regions and an object detection network (Fast R-CNN detector) that classifies proposed regions. Particularly, the RPN module adopts a fully-convolutional network that leverages rectangular object proposals, each with an abjectness score. This module tends to share computation with a Fast R-CNN object detection network, and both networks contain a common configuration of convolutional layers. Furthermore, anchors are used to associate a scale and aspect ratio with the region proposals. By default, three scales and three aspect ratios are used. Faster R-CNN is a general network for handling object detection, which is considered as object translation invariant with reduced model size. RPN is capable of thoroughly learning to propose regions by neural networks, it remains valid even when the model size substantially increases. Although it achieves good detection accuracy on a GPU, the average processing speed still needs further improvement, with only 5 FPS.

EfficientNET: EfficientNET [14] achieves good detection accuracy with computationally efficient. The proposed model uses a BiFPN (bidirectional feature pyramid network) to aggregate/combine features from various levels of the feature pyramid, improving the detection's accuracy. Then, it adopts a compound scaling method to optimize the model architecture and achieves a balance between accuracy and efficiency.

MobileNET: Sandler *et al.* [15] proposed the MobileNET model, aiming at decreasing the computational and memory cost of deep neural networks for mobile and embedded devices. It achieves high accuracy on image classification tasks and shows promising object detection results by incorporating inverted residuals and linear bottlenecks.

YOLO and YOLO Lite: YOLO (You Only Look Once) [16] is a viable object-detection model, which is capable of detecting objects at a faster speed compared with Faster R-CNN. YOLO aims to achieve real-time recognition by making predictions for bounding boxes and class labels simultaneously. The fastest version of YOLO can offer 45 FPS on GPUs. However, the original YOLO model is still computationally expensive and requires a powerful GPU to run in real-time. To tackle the issue, Adarsh *et al.* [17] proposed a light-version, called TinyYOLO, achieving up to 244 FPS on a PC with GPU and 13 FPS on a non-GPU laptop. Furthermore, YOLO Lite [12] was proposed with a lightweight version of the original YOLO model that can be executed on non-GPU devices such as smartphones and embedded computer systems. YOLO Lite was achieved by changing the convolutional layers in YOLO to depth-wise separable convolutions, reducing the required computation volumes by 80 percent without sacrificing accuracy. This model achieves comparable accuracy to the original YOLO model and is significantly faster, performing a speed of 21 FPS on a non-GPU device with low computing power.

III. Our Approach

A. Model Architecture

Our model aims to design a smaller, quantized version of the YOLO model for object recognition, which is suitable for real-time object detection tasks and optimized for deployment on IoT devices with limited computational resources. Fig. 1 entails the overall architecture, which follows a pattern of alternating convolutional layers, batch normalization layers, and max pooling layers that extract features from the input image, progressively reducing spatial dimensions through max pooling. The number of filters increases gradually, allowing the model to charaterize more complex and high-level features. The final detection layer utilizes the features from previous layers to conduct object detection through predicting bounding boxes and class probabilities, with the sigmoid function ensuring that the output values are in the range $[0, 1]$. This represents object probabilities and bounding box coordinates.

One significant modification in our QATFP-YOLO is the reduction of the model layers from 9 layers (in the TinyYOLOv2 model) and 7 layers (in the YOLO-Lite model) to 6 layers. This design choice, when compared with the others, was driven to create a lighter version with fewer parameters and computations. The idea is to push YOLO to the limit and determine if we can achieve more positive results in terms of performance and speed. With fewer layers, the model will have faster inference times, which is crucial in real-time applications. Table I shows the detailed layer-wise structures. Particularly, our model starts with a convolutional layer (conv1) that takes input images with three channels (RGB) and applies eight 3×3 filters with a stride of 1 and padding of 1. Then, it

Authorized licensed use limited to: Universidad de Sevilla. Downloaded on April 18,2026 at 11:40:52 UTC from IEEE Xplore. Restrictions apply.

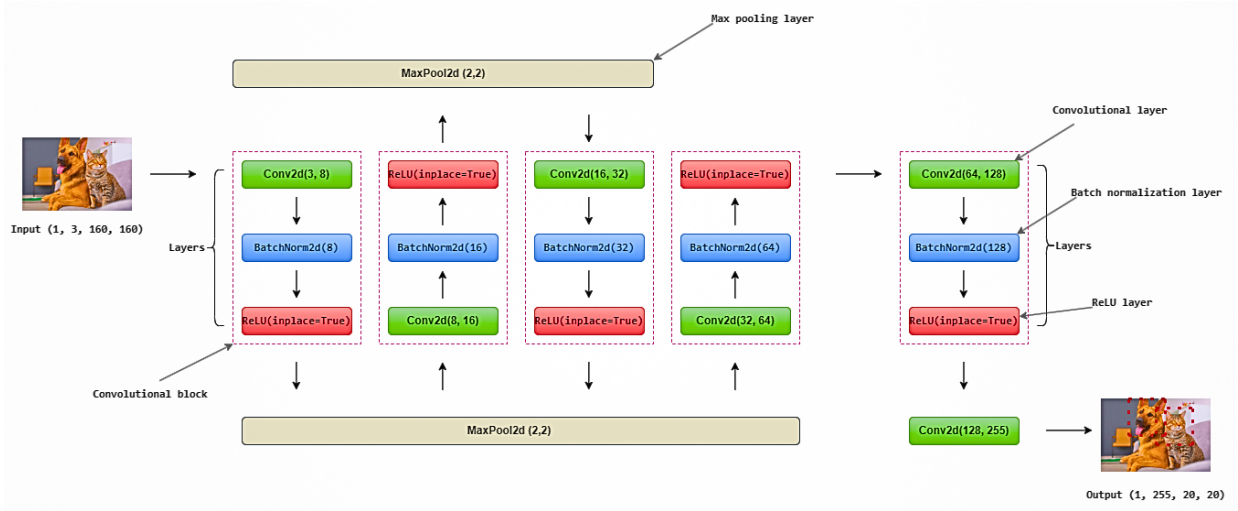


Fig. 1. Our Model Architecture

TABLE I. Breakdown of the Layers in Our Model's Structure

Layers	Layer Type	Input Channels	Output Channels	Kernel Sizes	Stride
1	Convolutional	3	8	3x3	1
	Batch Normalization	8	-	-	-
	Max Pooling	-	-	2x2	2
2	Convolutional	8	16	3x3	1
	Batch Normalization	16	-	-	-
	Max Pooling	-	-	2x2	2
3	Convolutional	16	32	3x3	1
	Batch Normalization	32	-	-	-
	Max Pooling	-	-	2x2	2
4	Convolutional	32	64	3x3	1
	Batch Normalization	-	-	-	-
5	Convolutional	64	128	3x3	1
	Batch Normalization	128	-	-	-
6	Convolutional	128	Num_anchors * (5 + num_classes)	1x1	1

is followed by batch normalization (bn1), a ReLU activation function (relu1), and a max-pooling layer (pool1) with a 2×2 kernel and stride of 2 reduces the spatial dimensions of the features. The process is repeated in the second layer with conv2, bn2, relu2, and pool2, but with 8 input channels and 16 output channels, and in the third layer with conv3, bn3, relu3, and pool3, using 16 input channels and 32 output channels, respectively. After that, we involve two more convolutional layers, e.g., conv4 and conv5, to further process the features with increasing numbers of output channels, e.g., 64 and 128, respectively. Each corresponding layer is followed by batch normalization and ReLU activation. Finally, the last convolutional layer, conv6, has 128 input channels and 255 output channels, corresponding to the number of classes or categories the model aims to predict. It uses a 1×1 kernel to generate a tensor representing the predicted class probabilities for each spatial location.

The overall architecture is designed to learn a hierarchical representation of the input image. In particular, lower-level convolutional blocks learn to extract low-level features (e.g., edges, corners) from the image. The max pooling layers then down-sample the feature maps while preserving the most critical information. The higher-level convolutional blocks then learn to extract complex features (e.g., object parts, shapes). Finally, the output layer combines the features from all the convolutional blocks to predict the class label of the image.

B. Optimization Approaches for Faster Inferences on Non-GPU Devices

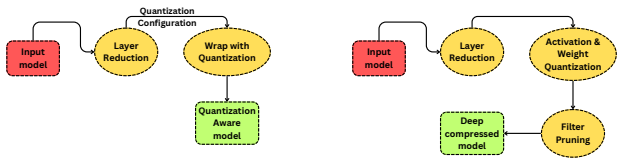
To enable fast inference, we consider involving two techniques to optimize our proposed model further, e.g., Quantization-Aware Training and Filter Pruning, elaborated in what follows.

Model Quantization and Quantization-Aware Training (QAT): Quantization is a technique employed to reduce the memory usage and computational requirements of neural networks without significantly sacrificing their accuracy [12]. The model underwent quantization using PyTorch's quantization module. Quantization is achieved by denoting the weights and activations of the model in a lower precision format, typically using fewer bits. This reduces the memory footprint required to store the model's parameters and results in faster computations during inference, particularly on hardware accelerators optimized for lower precision calculations.

The model was trained with simulated quantization effects, usually by introducing quantization-aware layers during training, known as quantization-aware training. This helps the model learn parameters that perform well under quantization. QAT is a training-time approach where the model is trained with quantization in mind, introducing quantization effects during training. Various layers are quantized, including convolutional and linear layers, which are the primary components responsible for most computations in the network.

During the initialization of the model, the quantization configuration is applied to specific layers using the qconfig attribute. This configuration defines the precision and other parameters for the quantization process, enabling fine-grained control over the quantization behavior. Applying quantization to the model reduces the precision of both weights and activations, typically from the full precision (e.g., 32-bit floating-point) to lower precision formats such as 8-bit integers. This

Authorized licensed use limited to: Universidad de Sevilla. Downloaded on April 18,2026 at 11:40:52 UTC from IEEE Xplore. Restrictions apply.



(a) 2-Stage Approach Featuring Quantization-Aware Training

(b) 3-Stage Approach Featuring QAT and Filter Pruning

Fig. 2. Our Proposed Two Optimization Strategies

reduction in precision allows for more efficient memory usage and faster computations. However, it is essential to note that quantization may result in a slight decrease in model accuracy.

Filter Pruning (FP): Each convolutional layer is identified, and 20 percent of the weights along with the channel dimension of each convolutional layer are pruned using ℓ_1-norm structured pruning, thereby reducing the model size and potentially improving computational efficiency during inference.

To achieve our goal of a faster inference speed on non-GPU devices, we design two optimization strategies: (i) *QAT:* This 2-stage method first reduces the layers in the model and then configures the model for Quantization aware training, as shown in Fig. 2. It assigns quantization configurations to the activation and weight observers for the convolutional and linear layers. The model is then wrapped to enable quantization during the forward pass. This allows the model to use quantized versions of activations and weight, enabling the model to perform quantized computations during training and inference. (ii) *QAT + FP:* This 3-stage deep compression method involves reduction of model layers, activation, and weight quantization followed by pruning the entire filters/channels rather than the individual weights, which is shown in Fig. 2. The pruning is performed along the channel dimension, and about 20 percent of the filters are to be pruned.

C. Data Processing and Model Training

Data Transformation: Before feeding the data into the model, multiple preprocessing steps are performed to improve its quality and facilitate effective model training. These steps are briefed as follows: (i) *Resizing:* To ensure a consistent input size for the model, we resize the images to a resolution of 160x160 pixels. This resizing step helps standardize the input format and introduces variability during training. Also, resizing the image to a consistent size ensures that all validation images have the same dimensions. (ii) *Random Horizontal Flip and Center Cropping:* Flipping the training images horizontally creates a mirrored version of the image to assist the model in improving its ability to deal with different object orientations. Cropping the center of the image helps maintain the critical features in the central region.

Conversion to Tensor: Deep learning models like YOLO work with tensors; therefore, converting the images to tensors is necessary to ensure model compatibility.

Normalization: To facilitate optimal model convergence during training, we normalize the pixel values of the images. This normalization step involves subtracting the mean value and then dividing by the standard deviation for each color channel (R, G, B). By normalizing the data, we mitigate the impact of varying pixel value ranges, stabilize the training process, and speed up convergence, which enables that the model can effectively learn from the features in the data.

Preprocessing Targets: YOLO models require a specific format for the ground truth annotations to effectively learn and forecast bounding box locations and class probabilities [12], thus making preprocessing the targets an essential aspect in training our model. This leads to the implementation of two functions used for processing the ground truth bounding box annotations for object detection in our model: (i) Intersection over Union (IoU) function and (ii) The targets preprocessing function. Here, The IoU function calculates the IoU between two bounding boxes, while the preprocess targets function converts the ground truth annotations into a tensor suitable for training our model. The preprocess target function prepares the ground truth annotations for training the model. It normalizes the bounding box coordinates, calculates the grid cell coordinates, and finds the best matching anchor box based on the IoU. It then updates the ground truth tensor with the information (e.g., bounding box coordinates, abjectness score, class probabilities). The tensor is returned after being moved to the specified device. The model's training is seamless by accurately processing ground truth bounding box annotations and generating the necessary tensors.

Training Procedure: During training, the network parameters are tuned for 20 epochs with Adam optimizer, where a learning rate ranging between 1e-4 and 5e-5 and batch sizes of 16 and 32. It adopts the Mean Squared Error (MSE) loss function and regularization techniques such as early stopping. Twelve different trials of the model are carried out, all exhibiting different hyperparameters. We initially implemented the model and used PyTorch to train it. We then converted it to TensorFlow Lite using ONNX. Various Quantization bits are also experimented on. Since the focus of this project is based on non-GPU, the model is trained on the CPU.

To realize accurate training, two key training techniques are considered: (i) *Decreasing the batch size*. It reduces the memory requirements during training, allowing for faster processing and potentially enabling the use of larger models. Additionally, smaller batch sizes can introduce stochasticity, which might help the model converge to better solutions. (ii) *Reducing the learning rate*. Lower learning rates allow the model to take smaller steps during optimization, helping it to fine-tune the weights and converge more precisely. However, using a lower learning rate may require more epochs to achieve a desirable level of performance.

Loss Function: The model incorporates the Mean Squared Error (MSE) loss function, which qualifies the mean squared difference between the predicted and target bounding box coordinates. The sigmoid activation function makes sure that the predicted bounding box coordinates fall in the range of [0,

Authorized licensed use limited to: Universidad de Sevilla. Downloaded on April 18,2026 at 11:40:52 UTC from IEEE Xplore. Restrictions apply.

1].

Optimizer: The Adam optimizer is utilized for updating the model's parameters during training process. It combines the advantages of adaptive gradient algorithms such as RMSprop, and stochastic gradient descent (SGD). It dynamically adjusts the learning rate based on the gradient's first and second moments, facilitating faster convergence.

Regularization Techniques: Early stopping is employed as a regularization technique during training. Early stopping prevents over-fitting by checking the model performance over a validation set during training. If the trained model's efficacy on the validation set cannot improve further or even begins to degrade, training will be halted early to avoid over-fitting the training data. The patience value is set to 5.

IV. Performance Evaluation

A. Methodology

Dataset Configurations: We train the model on the PascalVOC dataset [18], a widely used benchmark dataset in computer vision providing a comprehensive collection of images with annotated object bounding boxes. The dataset is divided into three subsets, e.g., training, validation, and testing, with sample sizes of 4008, 2510, and 1003, respectively.

Model Configurations: We consider varying the following hyper-parameters to check their effects on the model inference performance, e.g., learning rate, batch size, quantization bits, and filter pruning. We also compare our proposed 2-stage approach results with the 3-stage approach in optimizing the inference speed.

Evaluation on Mobile Devices: We consider deploying our AI solutions on a Google Pixel 4 smartphone to testify inference speed. Particularly, we utilize the AI Benchmark application [19]. It offers a platform to load custom TensorFlow Lite models and run them on Android devices with various acceleration options (Android NNAPI, TFLite GPU, etc.). It measures the inference speed, latency, initialization time, and real-time performance of models. Each of our designed models was converted from a Python machine learning model into a TensorFlow Lite model and implemented on a Google Pixel 4 smartphone with real-time inference on the built-in evaluation dataset in the app.

Evaluation Metrics in Inference Speed: We measure the inference speed using the FPS calculation, e.g., the number of frames (images) that the model can handle in one second. Also, we measure the average latency in a million seconds to process one-time inference on the mobile device.

B. Results

Inference Speed: After successfully training the models, the estimated FPS were evaluated using the formula, *FPS = Total images / Elapsed time*, where *Elapsed time* is the difference between *End time* and *Start time*. Table II shows the results. We find the best version of our designed models using our 2-stage optimization runs at approximately 88 FPS while the best version of our pruned model using our 3-stage optimization achieves an FPS of approximately 75 as shown in Fig. 3 and Fig. 4, correspondingly.

Results on Mobile Devices: Performance evaluation on

TABLE II. Performance Comparison of Different Models

Model	Number of Layers	Learning Rate	Batch Size	Pruning	Quantization	FPS	Inference Time (ms)
TinyYOLOv2	9	-	-	-	-	2.4	-
YOLO-Lite	7	-	-	yes	-	21	-
Our Model 1	6	1e-4	16	no	qint8	88.09	6.48
Our Model 2	6	1e-4	32	no	qint8	71.62	7.62
Our Model 3	6	5e-5	16	no	qint8	76.20	7.80
Our Model 4	6	5e-5	32	no	qint8	76.43	7.15
Our Model 5	6	1e-4	16	no	qint32	79.17	7.82
Our Model 6	6	1e-4	32	no	qint32	68.12	6.99
Our Model 7	6	5e-5	16	no	qint32	62.09	7.39
Our Model 8	6	5e-5	32	no	qint32	76.07	7.29
Our Model 9	6	1e-4	16	yes	qint8	74.50	8.20
Our Model 10	6	1e-4	32	yes	qint8	69.71	8.35
Our Model 11	6	5e-5	16	yes	qint8	70.95	8.42
Our Model 12	6	5e-5	32	yes	qint8	72.44	7.55

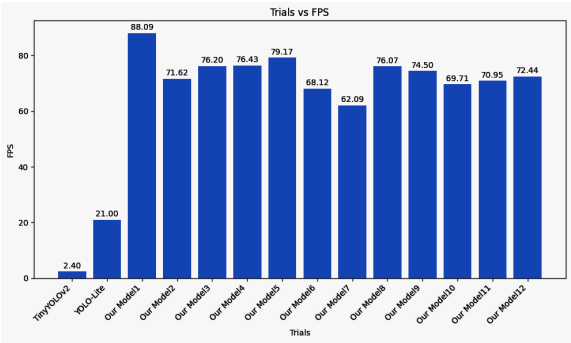


Fig. 3. Bar Plot of Estimated FPS

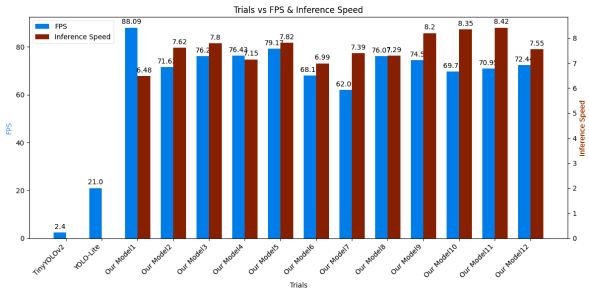


Fig. 4. Comparison of Trials Attempted When Developing QATFP-YOLO with Different Configurations

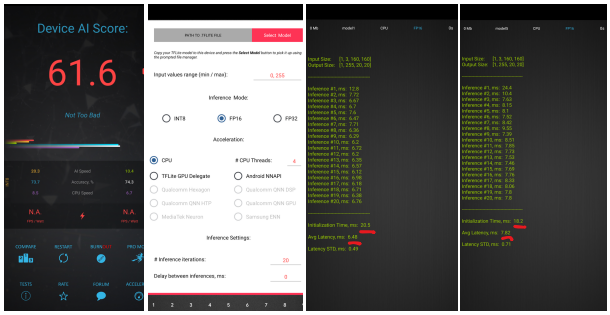


Fig. 5. An Illustration of Loading and Running Our Models on the AI Benchmark App, Followed by Inference Results of Different Models Evaluated on This App.

Authorized licensed use limited to: Universidad de Sevilla. Downloaded on April 18,2026 at 11:40:52 UTC from IEEE Xplore. Restrictions apply.

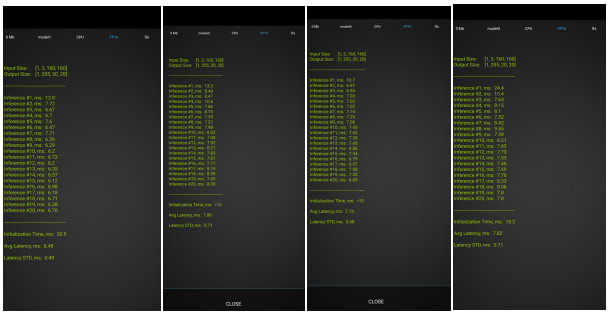


Fig. 6. Evaluation Results of Our Top Four Performing Models When Running on the AI Benchmark App.

mobile devices was conducted using the *AI Benchmark app* on a Google Pixel 4 phone. Various variants/trials of the optimized model are tested, and the results are shown in Fig. 5. The model was executed on a CPU using FP16 precision mode with an input size of 160x160 pixels and an output size of 20x20 pixels. The initialization time of the model was measured to be 20.5 ms, indicating the time taken to load the model into memory, which is reasonable for CPU execution. Inference time for each run ranged from 6.29 ms to 12.8 ms, with the model achieving an average latency as low as 6.48 ms, suggesting real-time inference capability. More details of the initialization time and inference latency on designed models are shown in Fig. 6.

Note that our proposed model outperformed the existing YOLO-Lite model by over fourfold in terms of inference speed, achieving 88 FPS. This is due to the reduction in model precision and size through quantization, leading to faster execution on mobile devices. After changing the precision of the model's parameters from floating-point to fixed-point arithmetic, the computational complexity of operations are reduced during inference. With fewer bits used to represent weights and activations, the arithmetic operations required for inference become simpler and faster to execute. This leads to a decrease in the time taken for computations, resulting in higher inference speeds. Also, due to the smaller model size, fewer memory accesses are required during inference, resulting in improved memory bandwidth utilization and reduced latency.

V. Final Remarks

In this paper, we introduced QATFP-YOLO, an optimized object detection model based on the benchmark YOLO model, to enhance inference speed on non-GPU devices. We proposed two optimization strategies: a 2-stage approach involving Quantization-Aware Training (QAT) and a 3-stage approach combining QAT with Filter Pruning (FT). The 2-stage method reduces computational complexity by employing smaller feature maps and fewer layers than YOLO-Lite, while the 3-stage method further enhances efficiency with filter pruning. We evaluated our model on the PascalVOC dataset. We deployed it on a real smartphone using an AI Benchmark app, achieving remarkable inference speeds of around 88 FPS, surpassing YOLO-Lite models by over fourfold.

References

[1] L. Jiao, F. Zhang, F. Liu, S. Yang, L. Li, Z. Feng, and R. Qu, "A survey of deep learning-based object detection," *IEEE Access*, vol. 7, pp. 128 837–128 868, 2019.

[2] Z.-Q. Zhao, P. Zheng, S.-T. Xu, and X. Wu, "Object detection with deep learning: A review," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 30, no. 11, pp. 3212–3232, 2019.

[3] T. Ji, Y. Guo, Q. Wang, X. Wang, and P. Li, "ECONOMY: Point clouds-based energy-efficient autonomous navigation for uavs," *IEEE Transactions on Network Science and Engineering*, vol. 8, no. 4, pp. 2885–2896, 2021.

[4] Y. Sun, F. Guo, F. Kaffashi, F. J. Jacono, M. DeGeorgia, and K. A. Loparo, "INSMA: An integrated system for multimodal data acquisition and analysis in the intensive care unit," *Journal of biomedical informatics*, vol. 106, p. 103434, 2020.

[5] W. G. Hatcher and W. Yu, "A survey of deep learning: Platforms, applications and emerging research trends," *IEEE Access*, vol. 6, pp. 24 411–24 432, 2018.

[6] F. Liang, W. G. Hatcher, W. Liao, W. Gao, and W. Yu, "Machine learning for security and the internet of things: The good, the bad, and the ugly," *IEEE Access*, vol. 7, pp. 158 126–158 147, 2019.

[7] Y. Wu, P. Tian, Y. Cao, L. Ge, and W. Yu, "Edge computing-based mobile object tracking in internet of things," *High-Confidence Computing*, vol. 2, no. 1, p. 100045, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2667295221000350

[8] C. Mukabe, N. Suresh, V. Hashiyana, T. Haiduwa, and W. Sverdlik, "Object detection and classification using machine learning techniques: A comparison of haar cascades and neural networks," in *Proceedings of the International Conference on Data Science, Machine Learning and Artificial Intelligence*, 2021, pp. 86–97.

[9] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems*, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, Eds., vol. 28. Curran Associates, Inc., 2015. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2015/file/14bfa6bb14875e45bba028a21ed38046-Paper.pdf

[10] R. Laroca, E. Severo, L. A. Zanlorensi, L. S. Oliveira, G. R. Gonçalves, W. R. Schwartz, and D. Menotti, "A robust real-time automatic license plate recognition based on the YOLO detector," in *2018 International Joint Conference on Neural Networks (IJCNN)*, 2018, pp. 1–10.

[11] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings I (14)*. Springer, 2016, pp. 21–37.

[12] R. Huang, J. Pedoeem, and C. Chen, "YOLO-LITE: A real-time object detection algorithm optimized for non-gpu computers," in *2018 IEEE International Conference on Big Data (Big Data)*, 2018, pp. 2503–2510.

[13] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," *International Conference on Learning Representations (ICLR)*, 2016.

[14] M. Tan and Q. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *International Conference on Machine Learning*. PMLR, 2019, pp. 6105–6114.

[15] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520.

[16] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.

[17] P. Adarsh, P. Rathi, and M. Kumar, "YOLO v3-Tiny: object detection and recognition using one stage improved model," in *2020 6th International Conference on Advanced Computing and Communication Systems (ICACCS)*, 2020, pp. 687–694.

[18] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman, "The pascal visual object classes (voc) challenge," *International journal of Computer Vision*, vol. 88, pp. 303–338, 2010.

[19] A. Ignatov, G. Malivenko, D. Plowman, S. Shukla, R. Timofte, Z. Zhang, Y. Wang, Z. Huang, G. Luo, G. Yu *et al.*, "Fast and accurate single-image depth estimation on mobile devices," *Mobile AI*, 2021.

Authorized licensed use limited to: Universidad de Sevilla. Downloaded on April 18,2026 at 11:40:52 UTC from IEEE Xplore. Restrictions apply.